Reviewer Pack — Tropicalized Brauer Groups and XOR-AND Tree Width

Entry 94c2421367c6 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 08:38:38 UTC

Tropicalized Brauer Groups and XOR-AND Tree Width

Entry ID: `94c2421367c6` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 08:38:38 UTC

1. Conjecture

Field A (mathematical branch): Algebraic K-theory (Brauer Groups) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

> ['For any Boolean function f, the degree of the Brauer group representation of its characteristic polynomial modulo 2 is bounded by the XOR-AND tree width of f.', 'Equivalently, for all boolean functions f with XOR-AND tree width w, there exists a representation of the characteristic polynomial of f such that its Brauer group degree is at most 2^w.']

Rationale (proposer's reasoning):

> ['Brauer groups provide a measure of non-triviality in algebraic K-theory and can be used to study the structure of field extensions. The XOR-AND tree width captures the complexity of evaluating boolean functions, suggesting that Brauer group analysis could reveal deeper structures related to computational complexity.']

Taxonomy category: `Algebraic_K_theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `141f62ef244fef67`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the correlation coefficient between the degree of the Brauer group representation and the XOR-AND tree width for all tested functions is ≥ 0.7, with p-value ≤ 0.05. The conjecture is falsified if this correlation coefficient is < 0.5 or p-value > 0.2.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.80	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `NOVEL` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'algebraic K-theory Brauer groups' AND 'XOR-AND tree width'` - `'characteristic polynomial modulo 2' AND 'Brauer group representation' AND 'XOR-AND tree width'` - `'Boolean function' AND 'degree of Brauer group' AND 'XOR-AND tree width'`

Top relevant hits considered: - [http://arxiv.org/abs/1311.1421v3] Multiplicative differential algebraic K-theory and applications - [http://arxiv.org/abs/2104.04021v2] A universal characterization of noncommutative motives and secondary algebraic K-theory - [http://arxiv.org/abs/1811.09564v3] Dévissage for Waldhausen K-theory - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/2308.14302v2] Finite simple characteristic quotients of the free group of rank 2 - [http://arxiv.org/abs/2306.12196v1] Probabilistic estimation of the algebraic degree of Boolean functions - [http://arxiv.org/abs/1402.5652v3] Compact presentability of tree almost automorphism groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def xor_and_tree_width(f):
        if len(f) == 1:
            return 0
        else:
            left = f[:len(f)//2]
            right = f[len(f)//2:]
            return max(xor_and_tree_width(left), xor_and_tree_width(right)) + 1

    def characteristic_polynomial(f):
        n = len(f)
        if n == 1:
            return [f[0]]
        else:
            left_poly = characteristic_polynomial(f[:n//2])
            right_poly = characteristic_polynomial(f[n//2:])
            result = [0] * (len(left_poly) + len(right_poly) - 1)
            for i in range(len(left_poly)):
                for j in range(len(right_poly)):
```

```
                    result[i+j] += left_poly[i] * right_poly[j]
            return result

    def brauer_group_degree(poly):
        n = len(poly)
        if n == 1:
            return 0
        else:
            degree = 0
            for i in range(1, n):
                if poly[i] != 0:
                    degree += 1
            return degree

    max_n = 40
    degrees = []
    widths = []

    for _ in range(30):  # Ensure at least 30 instances per seed
        n = random.randint(5, max_n)
        f = [random.choice([0, 1]) for _ in range(n)]
        width = xor_and_tree_width(f)
        poly = characteristic_polynomial(f)
        degree = brauer_group_degree(poly)

        degrees.append(degree)
        widths.append(width)

    correlation_coefficient = sum((d - mean_d) * (w - mean_w) for d, w in zip(degrees, widths)) /
    p_value = 2 * (1 - abs(correlation_coefficient))

    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": n_tested,
        "conjecture_holds": correlation_coefficient >= 0.7 and p_value <= 0.05,
        "counterexample": "" if correlation_coefficient >= 0.7 and p_value <= 0.05 else f"correlat
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else list(range(2, 30))  # Default

    results = []
    n_tested = 0

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
        n_tested += result["instances_tested"]

    mean_d = sum(r["metric_value"] for r in results) / len(results)
    std_d = (sum((r["metric_value"] - mean_d)**2 for r in results) / len(results))**0.5

    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)
```

```
    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_d} std={std_d} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{results[first_failing_seed]['counterexample']}
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2d36145f.py", line 86, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2d36145f.py", line 67, in run_trial
    correlation_coefficient = sum((d - mean_d) * (w - mean_w) for d, w in zip(degrees, widths)) / (n_tes
                              ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_2d36145f.py", line 67, in <genexpr>
    correlation_coefficient = sum((d - mean_d) * (w - mean_w) for d, w in zip(degrees, widths)) / (n_tes
                                   ^^^^^^
NameError: name 'mean_d' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test code crashed before producing data, which prevents us from evaluating the correlation coefficient and p-value required to support or falsify the conjecture. | next: Investigate the cause of the crash in the test code and run the test again to obtain the necessary data for a proper evaluation.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11748
2	propose	ollama_remote	glm4:latest	0	0	9585
3	preregistration	ollama_remote	glm4:latest	0	0	5699
4	novelty	ollama_remote	glm4:latest	0	0	5103

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	novelty	ollama_remote	glm4:latest	0	0	9047
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15800
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12018
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14106
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10213
10	judge	ollama_remote	glm4:latest	0	0	8564

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 101883 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/94c2421367c6.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/94c2421367c6.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/94c2421367c6.tar.gz` (if generated)